# DSC630 Milestone 5

March 7, 2026

## 1 Milestone 5

### 1.1 DSC630 Ayden Trusty

;========================================== */

Title: Milestone 5

Author(s): Abbott, Dean. Applied Predictive Analytics (Textbook)

Date: March 7 2026

Modified By: Ayden Trusty

Description: This program is to refresh my understanding/practice of using Python and/or R for milestone 3.

**Dataset: https://www.kaggle.com/datasets/uciml/default-of-credit-card-clients-dataset?resource=download**

**1. Import packages**

[5]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    roc_auc_score, RocCurveDisplay
)
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

**2. Fix Column Issues and Identify the Target Variable** I cleaned the column names to remove any extra spaces and dropped the ID column because it does not contribute to predicting credit default. I then identified the target variable representing whether a customer defaulted and ensured it was stored as a numeric value so the machine learning models could use it.

[6]:
```
DATA_PATH = "UCI_Credit_Card.csv"

try:
    df = pd.read_csv(DATA_PATH)
except Exception:
    df = pd.read_excel(DATA_PATH)

print("Shape:", df.shape)
print(df.head())
```

```
Shape: (30000, 25)
   ID  LIMIT_BAL  SEX  EDUCATION  MARRIAGE  AGE  PAY_0  PAY_2  PAY_3  PAY_4  \
0   1    20000.0    2          2         1   24      2      2     -1     -1
1   2   120000.0    2          2         2   26     -1      2      0      0
2   3    90000.0    2          2         2   34      0      0      0      0
3   4    50000.0    2          2         1   37      0      0      0      0
4   5    50000.0    1          2         1   57     -1      0     -1      0

   …  BILL_AMT4  BILL_AMT5  BILL_AMT6  PAY_AMT1  PAY_AMT2  PAY_AMT3  \
0  …        0.0        0.0        0.0       0.0     689.0       0.0
1  …     3272.0     3455.0     3261.0       0.0    1000.0    1000.0
2  …    14331.0    14948.0    15549.0    1518.0    1500.0    1000.0
3  …    28314.0    28959.0    29547.0    2000.0    2019.0    1200.0
4  …    20940.0    19146.0    19131.0    2000.0   36681.0   10000.0

   PAY_AMT4  PAY_AMT5  PAY_AMT6  default.payment.next.month
0       0.0       0.0       0.0                          1
1    1000.0       0.0    2000.0                          1
2    1000.0    1000.0    5000.0                          0
3    1100.0    1069.0    1000.0                          0
4    9000.0     689.0     679.0                          0

[5 rows x 25 columns]
```

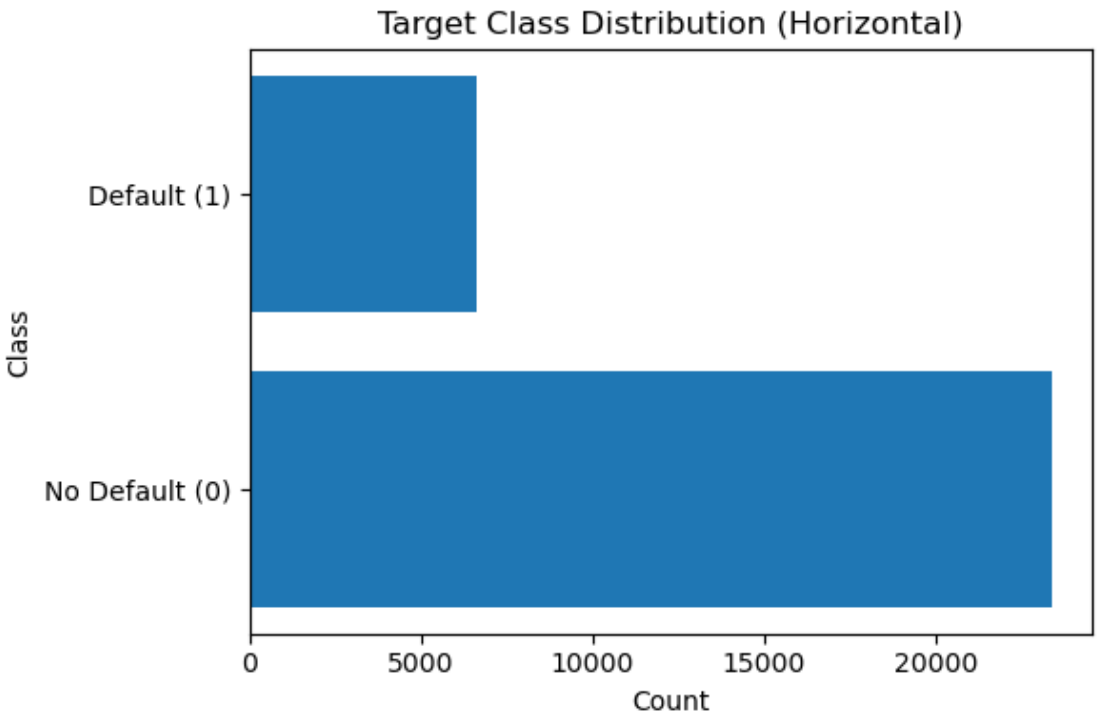
**3. Explore Class Distribution (Imbalance Check)** I examined the distribution of the target variable to determine whether the dataset is imbalanced. This step is important because if one class dominates, a model may appear accurate while failing to detect the minority class, which in this case represents default risk.

[19]:
```
# Some versions have an unnamed first column (ID index)
# or column names with leading/trailing spaces
df.columns = [c.strip() for c in df.columns]

# If there's an "ID" column, it's not predictive - drop it
if "ID" in df.columns:
    df = df.drop(columns=["ID"])
```

```
# Target name differs slightly depending on dataset version
possible_targets = [
    "default.payment.next.month",
    "default payment next month",
    "DEFAULT",  # some variants
]
target_col = None
for c in possible_targets:
    if c in df.columns:
        target_col = c
        break

if target_col is None:
    raise ValueError(f"Could not find target column. Available columns: {df.
 ↪columns.tolist()}")

print("Target column:", target_col)

# Make sure target is 0/1 integer
df[target_col] = df[target_col].astype(int)
```

```
Target column: default.payment.next.month
```

[20]:
```
plt.figure(figsize=(6, 4))
plt.barh(["No Default (0)", "Default (1)"], counts.sort_index().values)
plt.title("Target Class Distribution (Horizontal)")
plt.xlabel("Count")
plt.ylabel("Class")
plt.tight_layout()
plt.show()
```



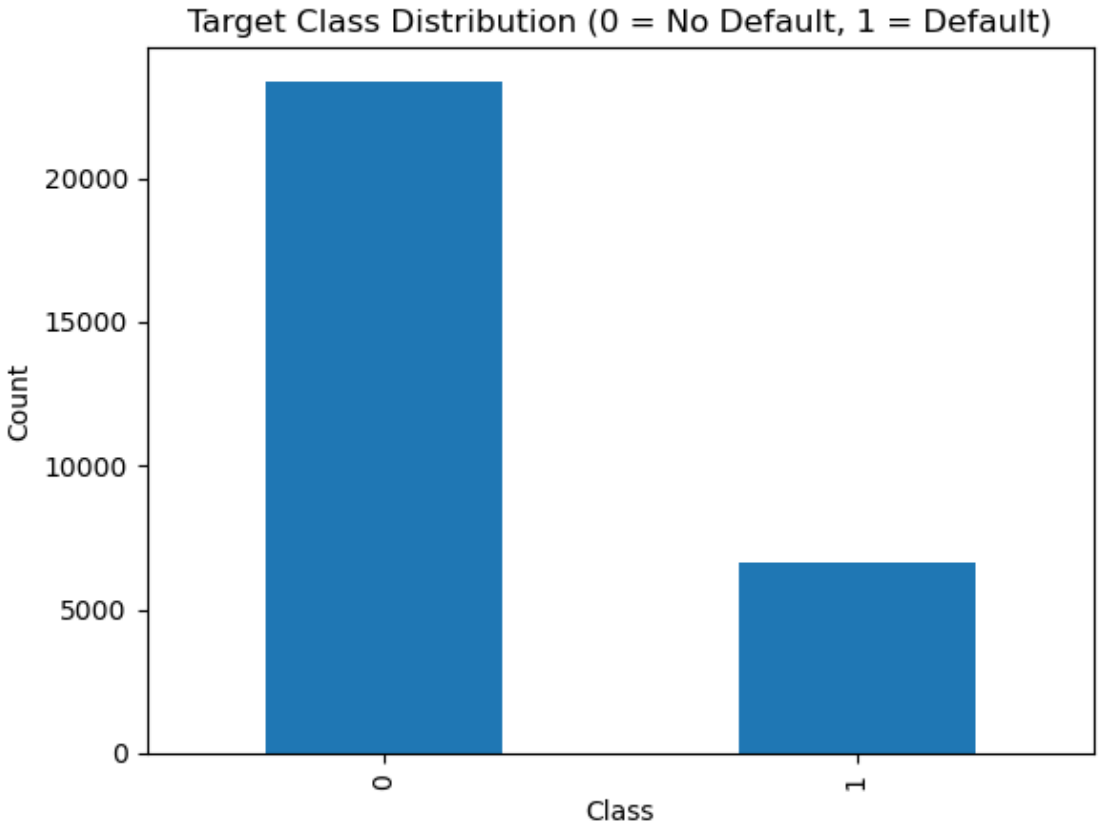

[ ]:

**4. Check for Missing Values** I checked for missing values to assess data quality. Missing data can distort analysis and negatively affect model performance, so this step ensures the dataset is suitable for modeling or indicates if additional cleaning is required.

[21]:
```
counts = df[target_col].value_counts()
props = df[target_col].value_counts(normalize=True)

print("\nClass counts:\n", counts)
print("\nClass proportions:\n", props)

plt.figure()
counts.sort_index().plot(kind="bar")
plt.title("Target Class Distribution (0 = No Default, 1 = Default)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```

```
Class counts:
 default.payment.next.month
0    23364
1     6636
```

```
Name: count, dtype: int64

Class proportions:
 default.payment.next.month
0    0.7788
1    0.2212
Name: proportion, dtype: float64
```




**5. Feature overview** I just wanted to see how many missing values exist.

[22]:
```
missing = df.isna().sum().sort_values(ascending=False)
print("\nTop missing values:\n", missing.head(10))
```

```
Top missing values:
 LIMIT_BAL    0
SEX           0
PAY_AMT6      0
PAY_AMT5      0
PAY_AMT4      0
PAY_AMT3      0
```

```
PAY_AMT2     0
PAY_AMT1     0
BILL_AMT6    0
BILL_AMT5    0
dtype: int64
```

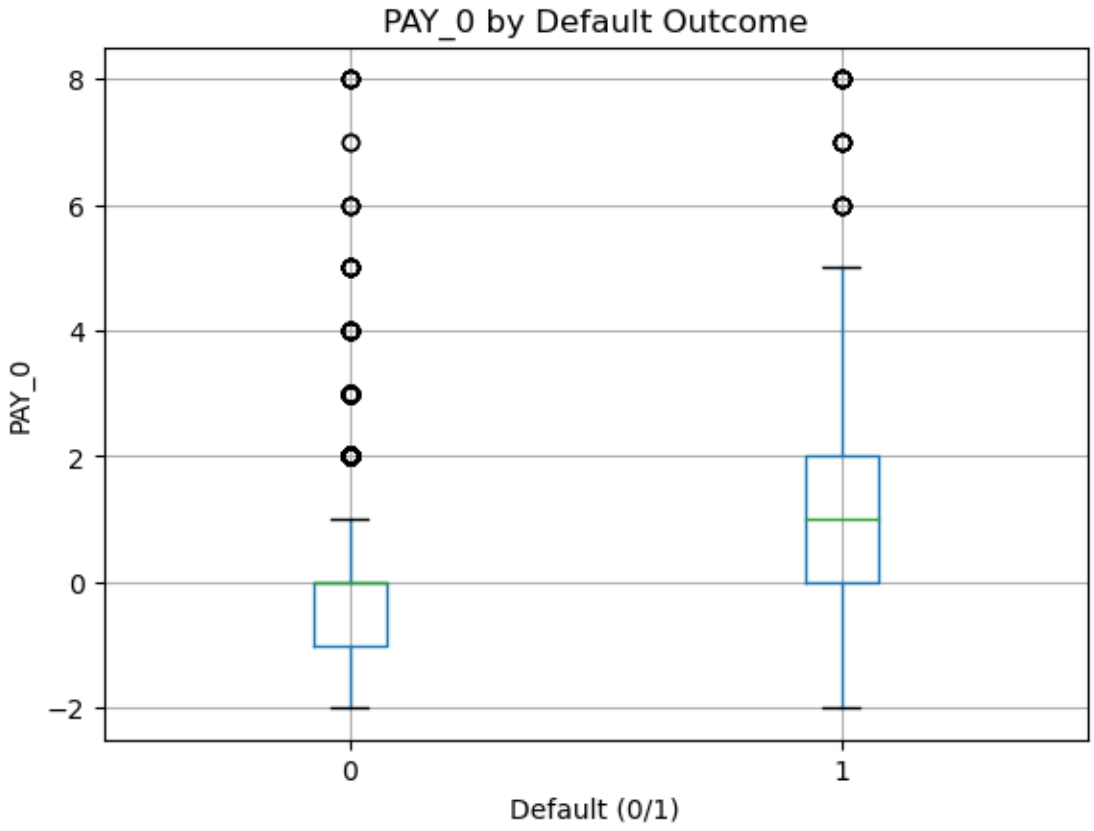
**6. Plots** This entire section is more so just exploring what kind of plots to use and how they could be useful for my project.

[23]:
```
print("\nSummary stats (numeric):")
print(df.describe(include=[np.number]).T.head(15))
```

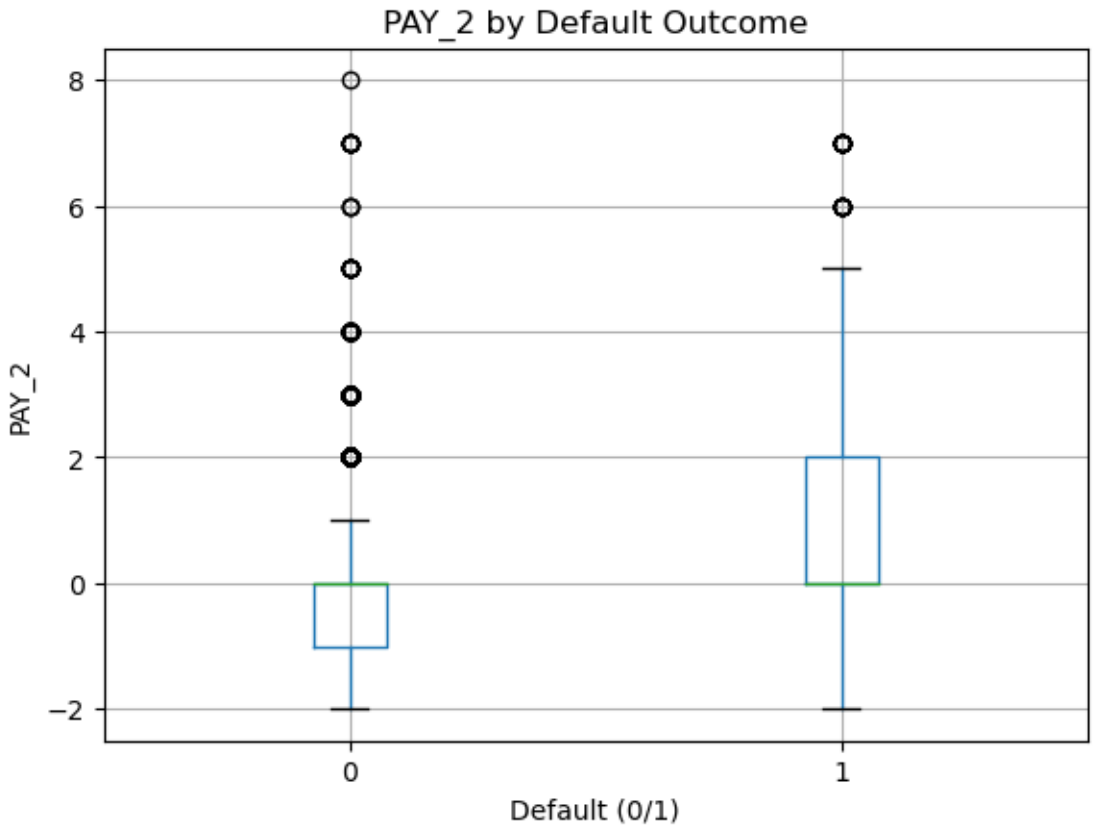
```
Summary stats (numeric):
              count           mean             std       min       25%  \
LIMIT_BAL   30000.0  167484.322667  129747.661567   10000.0  50000.00
SEX         30000.0       1.603733       0.489129       1.0      1.00
EDUCATION   30000.0       1.853133       0.790349       0.0      1.00
MARRIAGE    30000.0       1.551867       0.521970       0.0      1.00
AGE         30000.0      35.485500       9.217904      21.0     28.00
PAY_0       30000.0      -0.016700       1.123802      -2.0     -1.00
PAY_2       30000.0      -0.133767       1.197186      -2.0     -1.00
PAY_3       30000.0      -0.166200       1.196868      -2.0     -1.00
PAY_4       30000.0      -0.220667       1.169139      -2.0     -1.00
PAY_5       30000.0      -0.266200       1.133187      -2.0     -1.00
PAY_6       30000.0      -0.291100       1.149988      -2.0     -1.00
BILL_AMT1   30000.0   51223.330900   73635.860576 -165580.0   3558.75
BILL_AMT2   30000.0   49179.075167   71173.768783  -69777.0   2984.75
BILL_AMT3   30000.0   47013.154800   69349.387427 -157264.0   2666.25
BILL_AMT4   30000.0   43262.948967   64332.856134 -170000.0   2326.75

                50%        75%        max
LIMIT_BAL  140000.0  240000.00  1000000.0
SEX             2.0       2.00        2.0
EDUCATION       2.0       2.00        6.0
MARRIAGE        2.0       2.00        3.0
AGE            34.0      41.00       79.0
PAY_0           0.0       0.00        8.0
PAY_2           0.0       0.00        8.0
PAY_3           0.0       0.00        8.0
PAY_4           0.0       0.00        8.0
PAY_5           0.0       0.00        8.0
PAY_6           0.0       0.00        8.0
BILL_AMT1   22381.5   67091.00   964511.0
BILL_AMT2   21200.0   64006.25   983931.0
BILL_AMT3   20088.5   60164.75  1664089.0
BILL_AMT4   19052.0   54506.00   891586.0
```

[24]:
```
# 6a) Repayment status features are usually named PAY_0, PAY_2, ... PAY_6
pay_cols = [c for c in df.columns if c.startswith("PAY_")]
if pay_cols:
    for c in sorted(pay_cols):
        plt.figure()
        df.boxplot(column=c, by=target_col)
        plt.title(f"{c} by Default Outcome")
        plt.suptitle("")  # remove auto title
        plt.xlabel("Default (0/1)")
        plt.ylabel(c)
        plt.show()
```

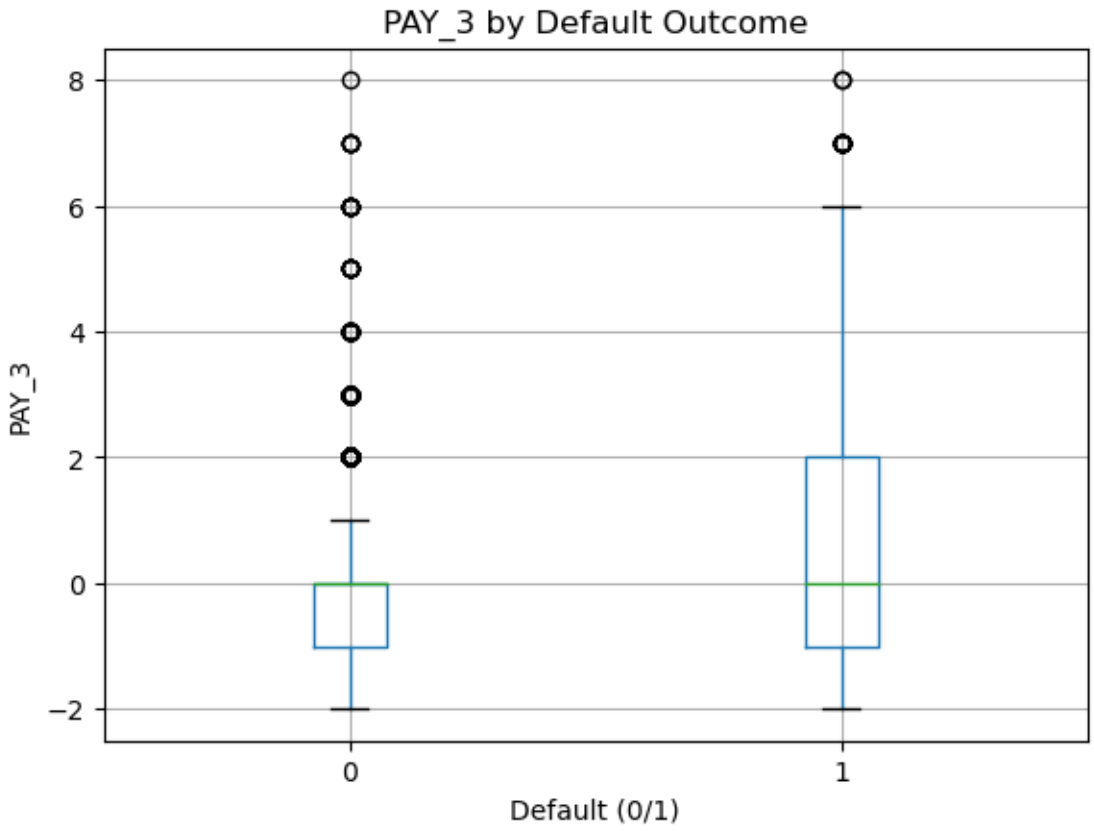
```
<Figure size 640x480 with 0 Axes>
```



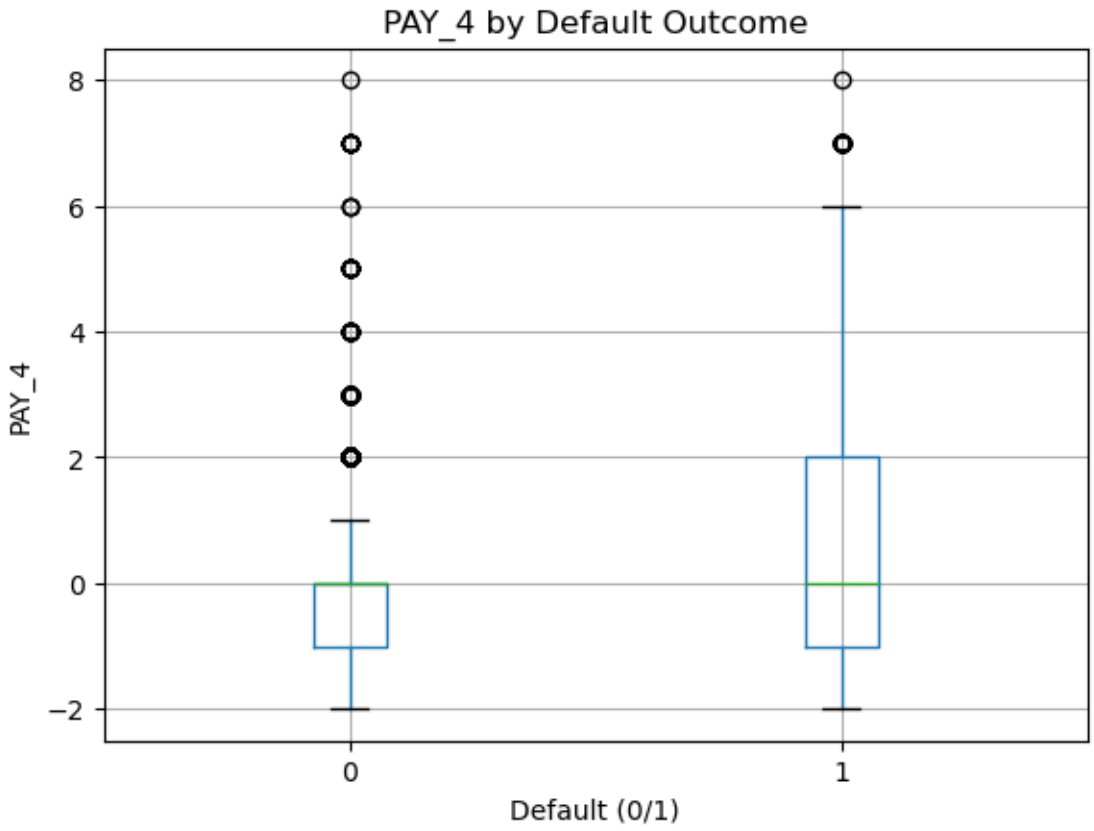

```
<Figure size 640x480 with 0 Axes>
```



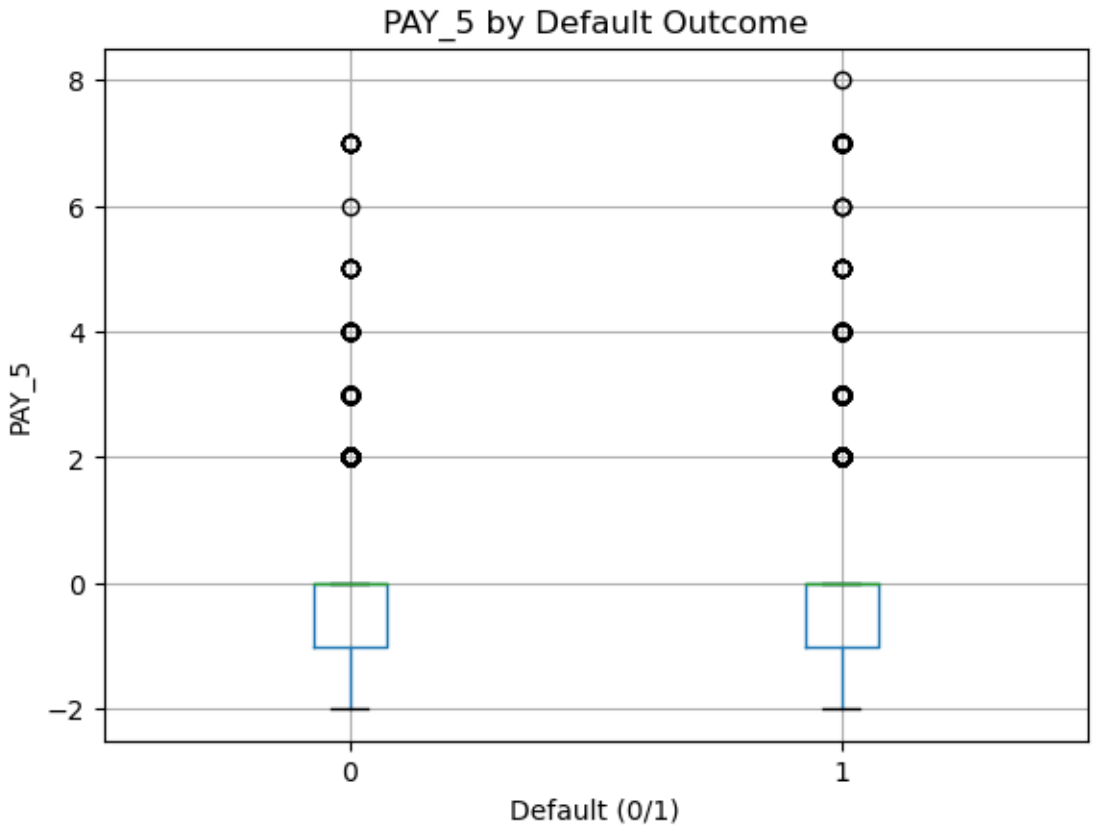

```
<Figure size 640x480 with 0 Axes>
```



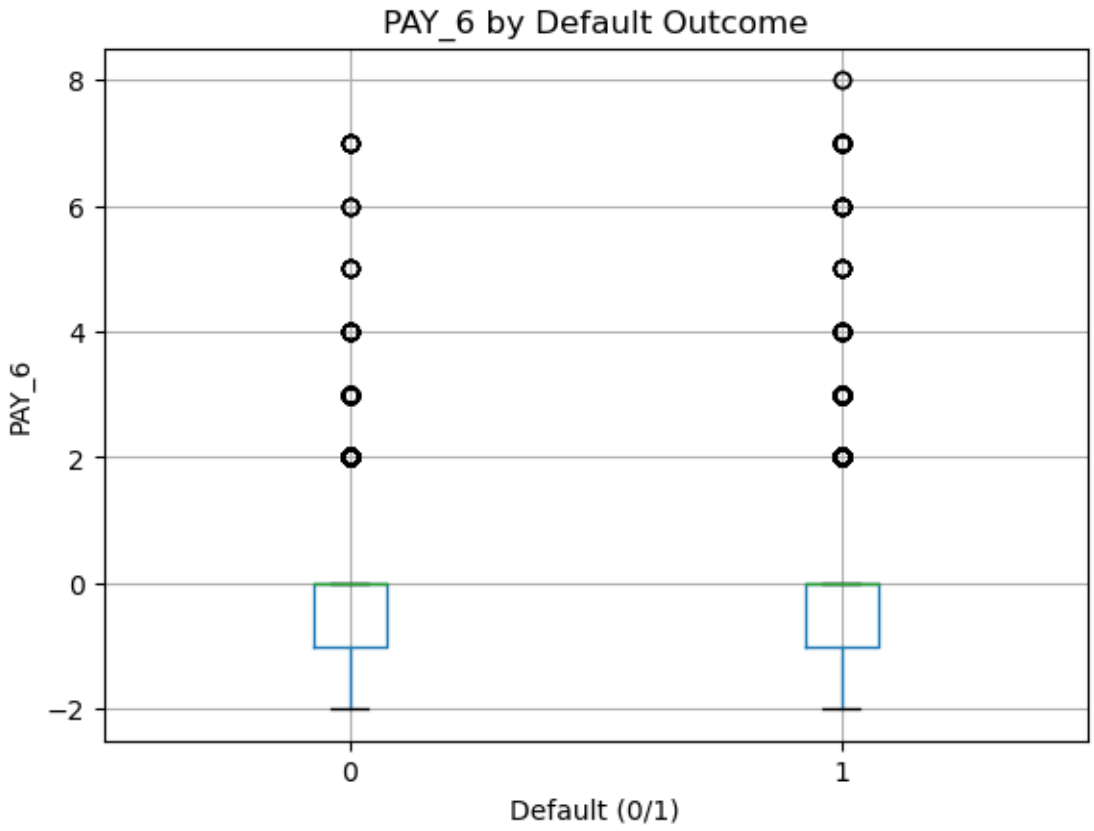

```
<Figure size 640x480 with 0 Axes>
```



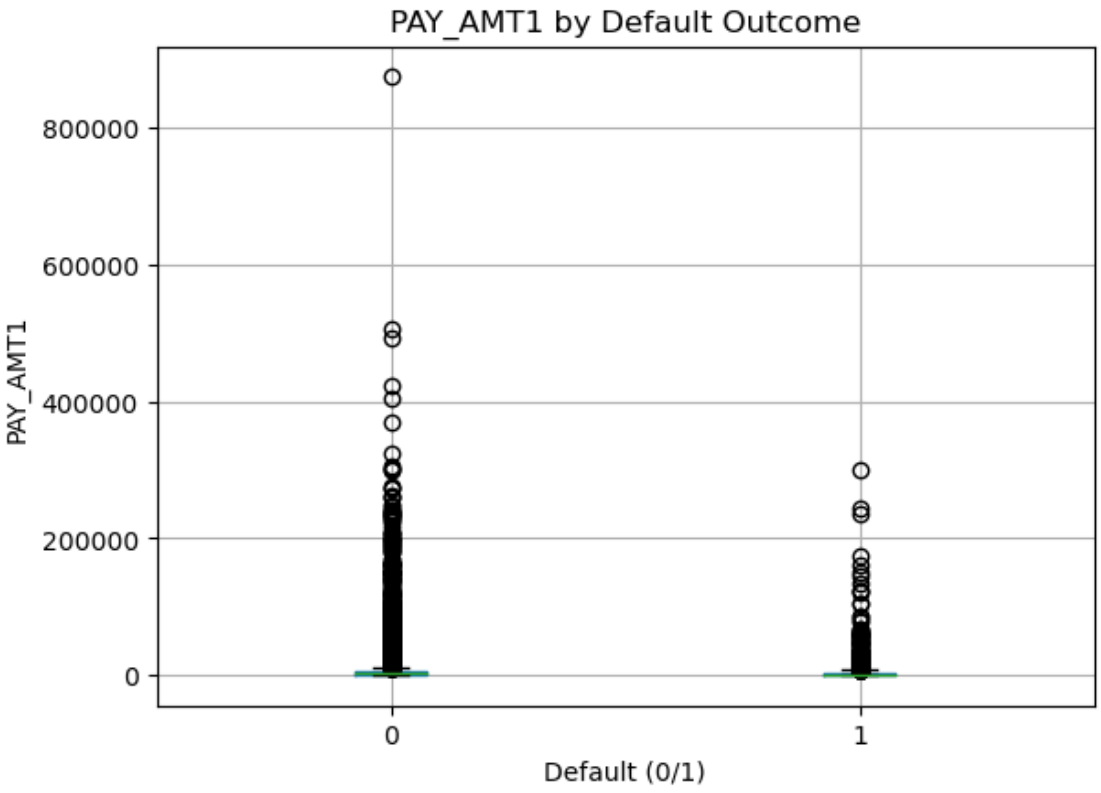

```
<Figure size 640x480 with 0 Axes>
```



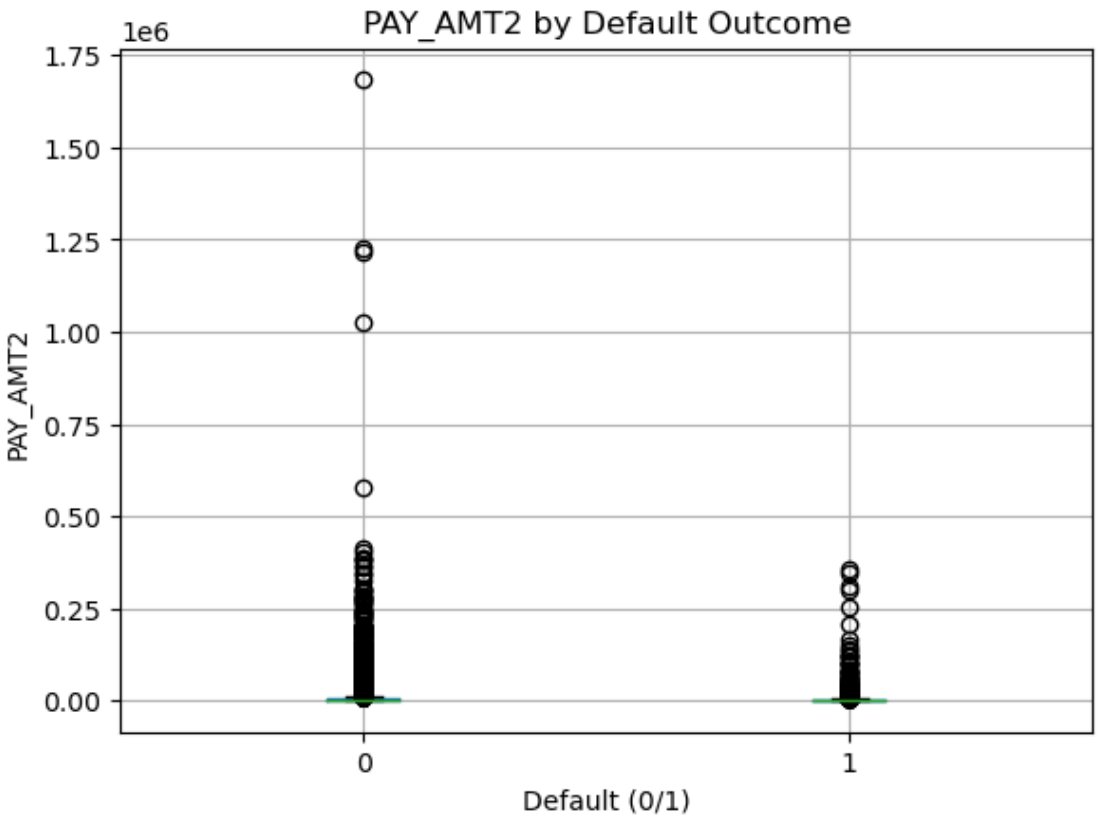

```
<Figure size 640x480 with 0 Axes>
```



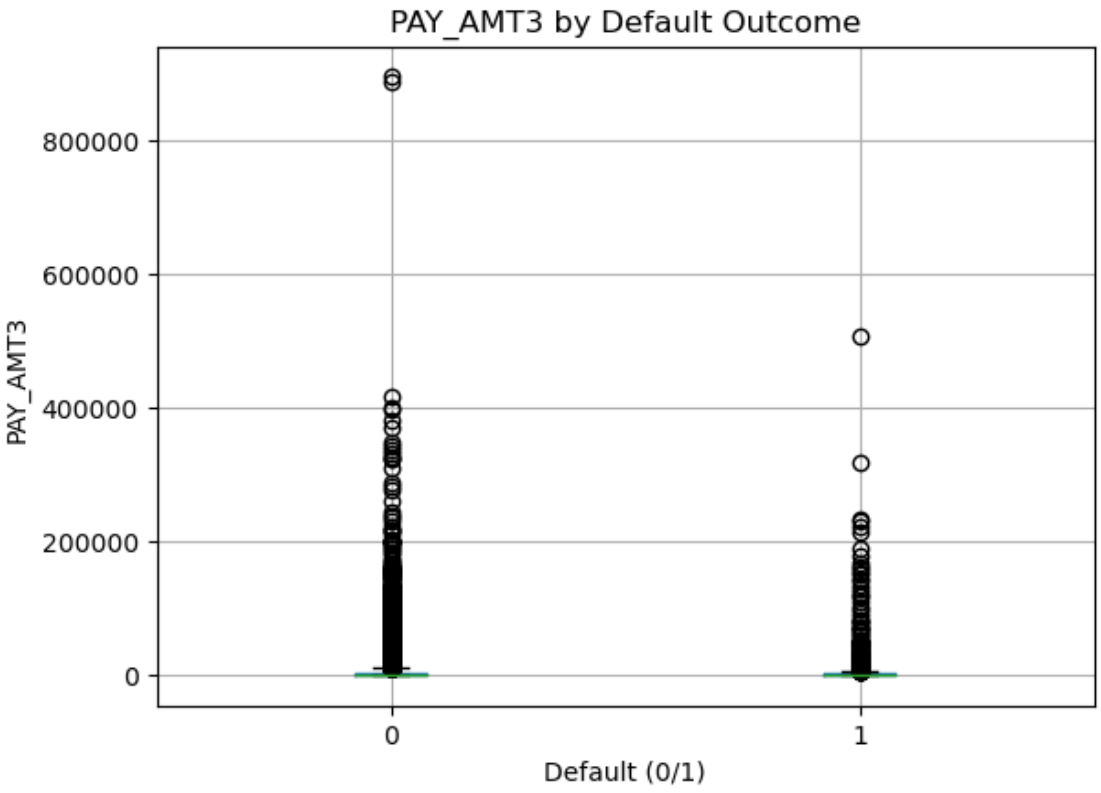

```
<Figure size 640x480 with 0 Axes>
```



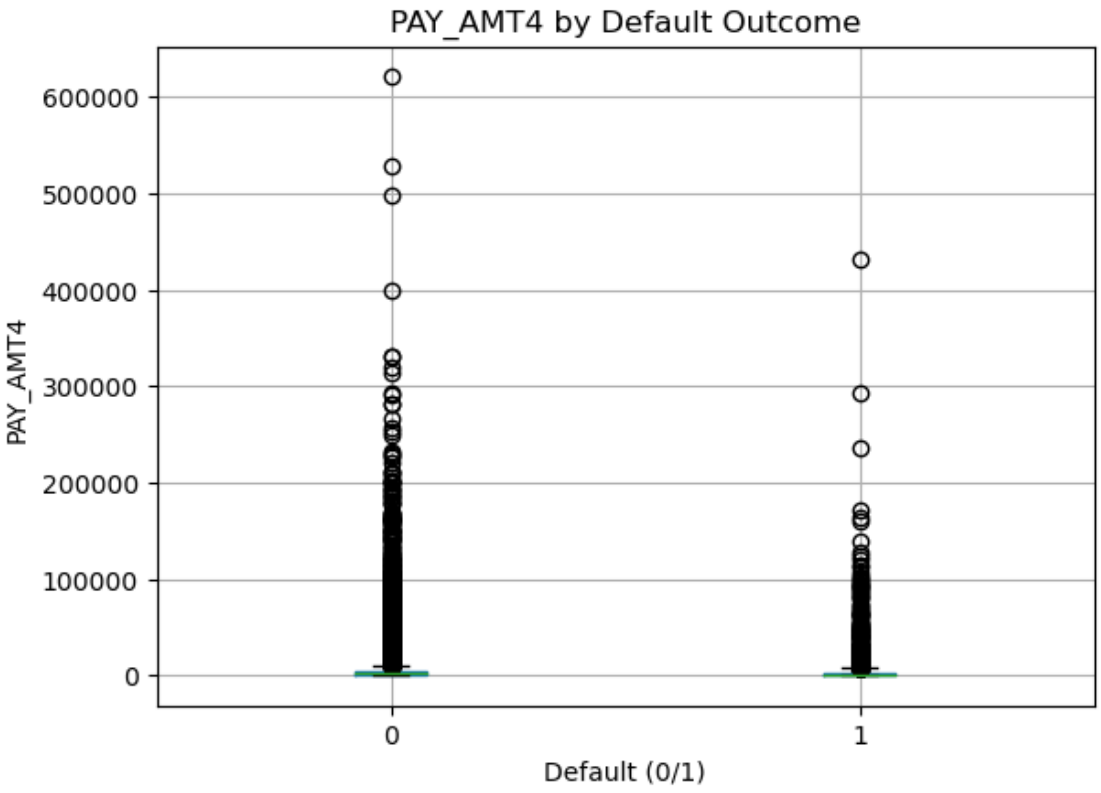

```
<Figure size 640x480 with 0 Axes>
```



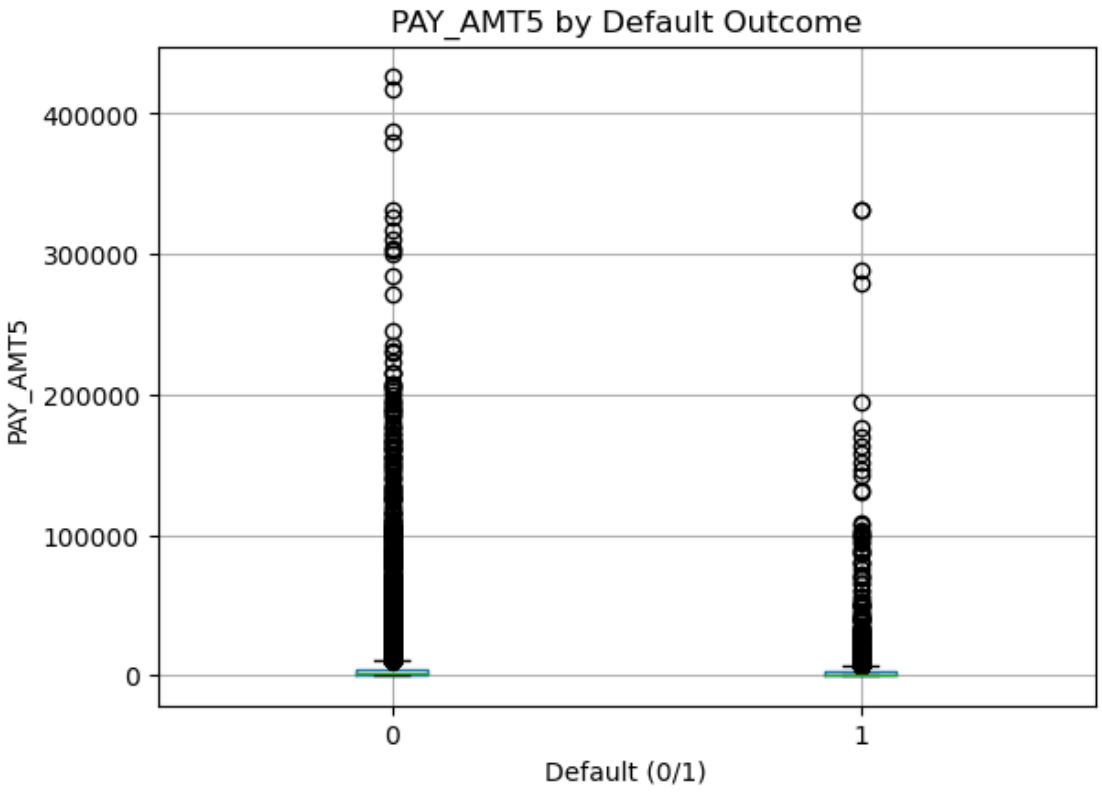

```
<Figure size 640x480 with 0 Axes>
```



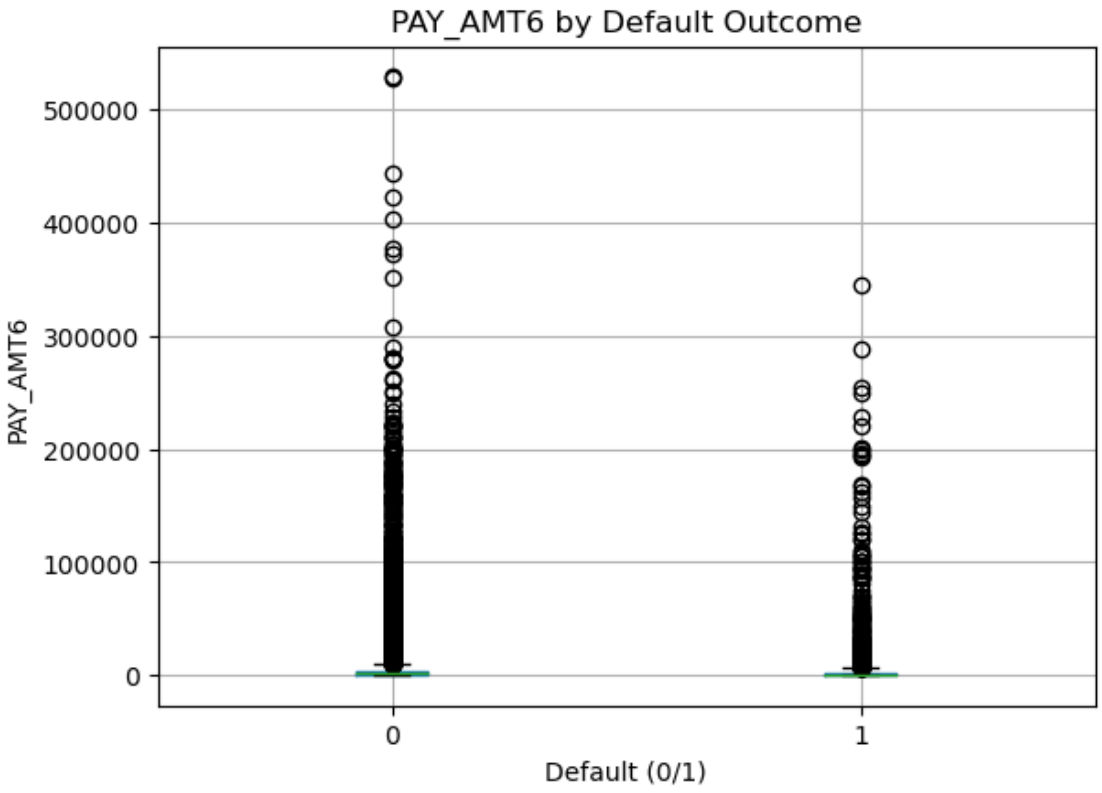

```
<Figure size 640x480 with 0 Axes>
```



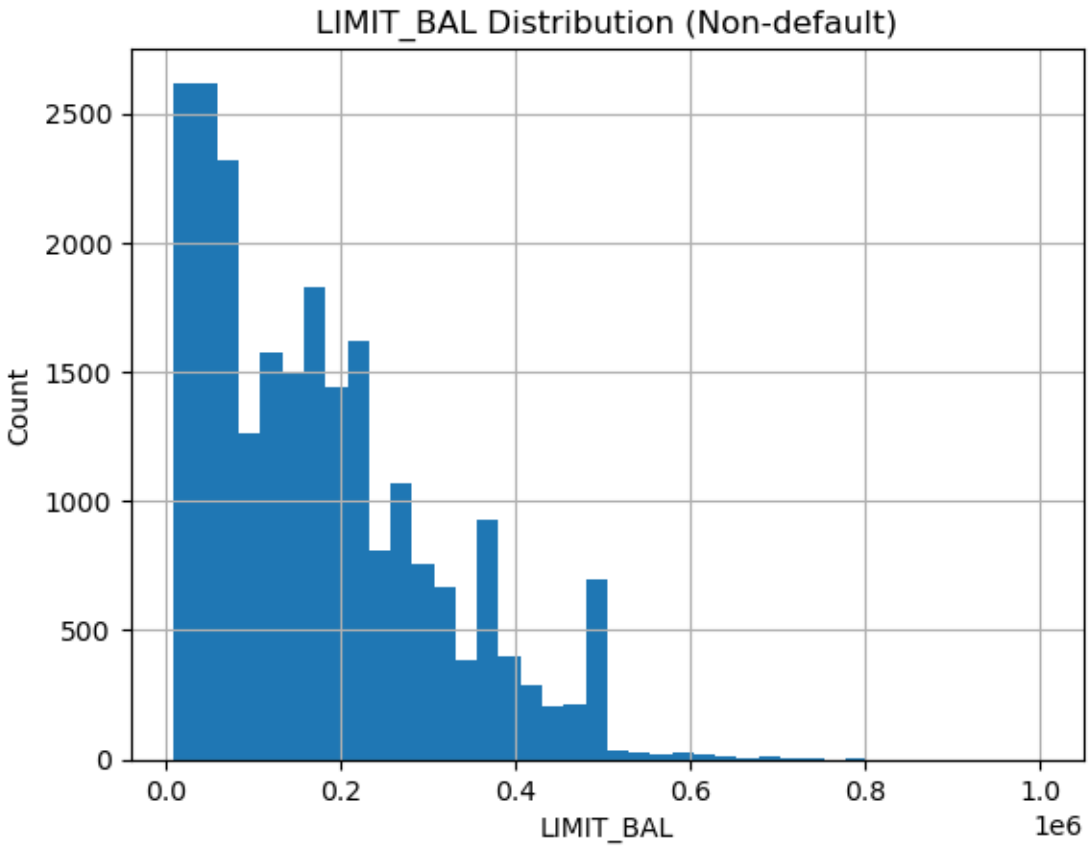

<Figure size 640x480 with 0 Axes>

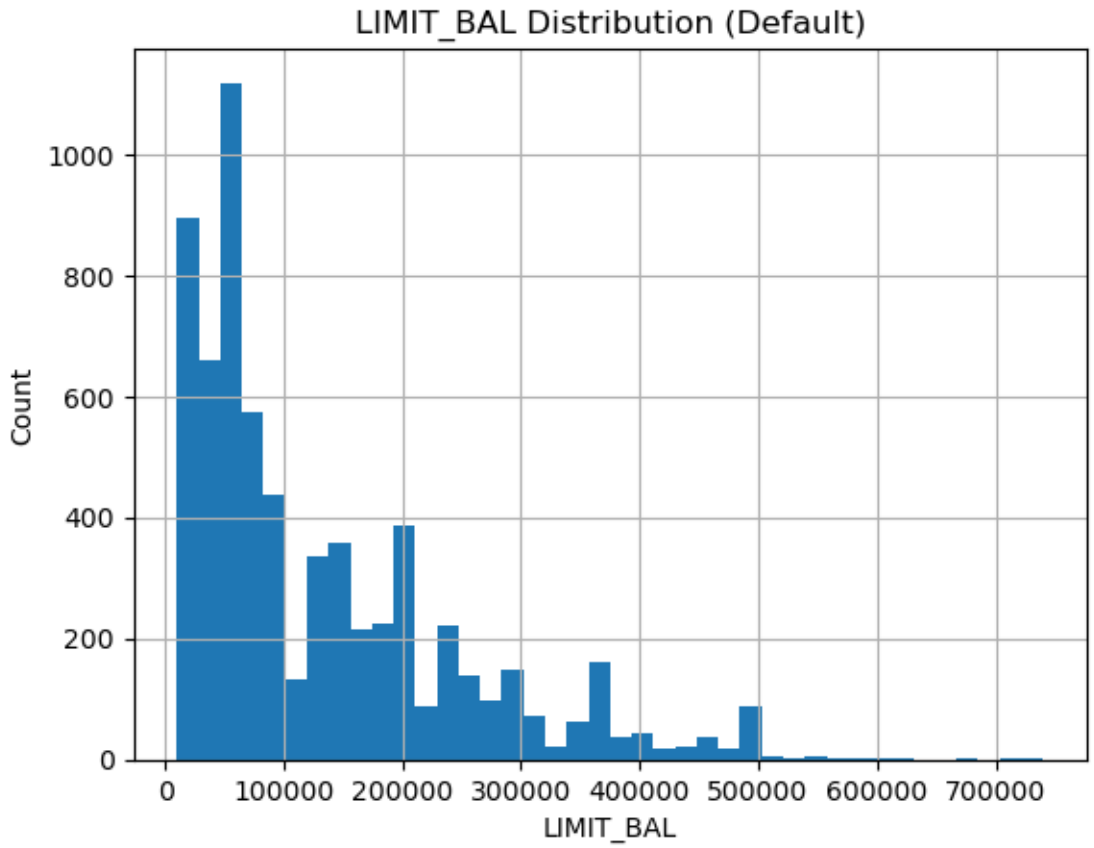



```
<Figure size 640x480 with 0 Axes>
```

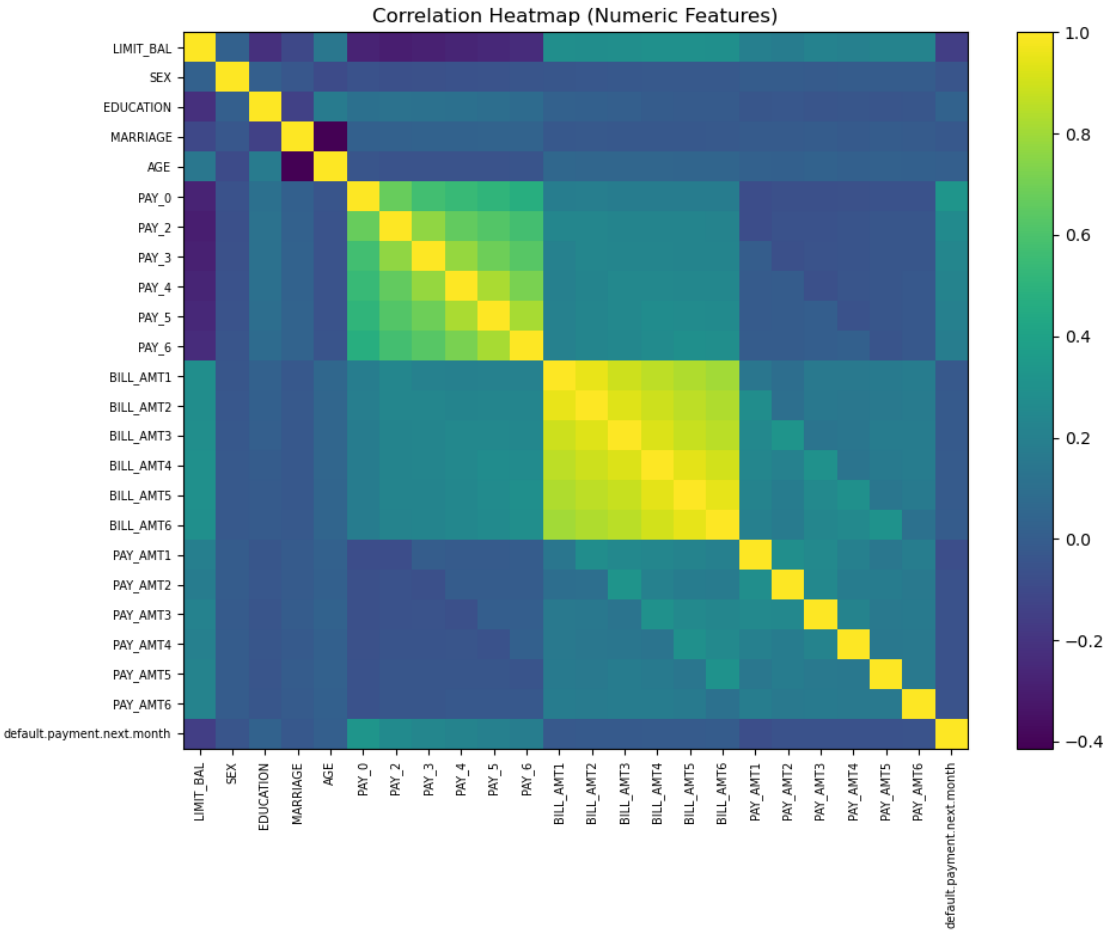



[25]:
```
# 6b) Credit limit (LIMIT_BAL)
if "LIMIT_BAL" in df.columns:
    plt.figure()
    df[df[target_col] == 0]["LIMIT_BAL"].hist(bins=40)
    plt.title("LIMIT_BAL Distribution (Non-default)")
    plt.xlabel("LIMIT_BAL")
    plt.ylabel("Count")
    plt.show()

    plt.figure()
    df[df[target_col] == 1]["LIMIT_BAL"].hist(bins=40)
    plt.title("LIMIT_BAL Distribution (Default)")
    plt.xlabel("LIMIT_BAL")
    plt.ylabel("Count")
    plt.show()
```

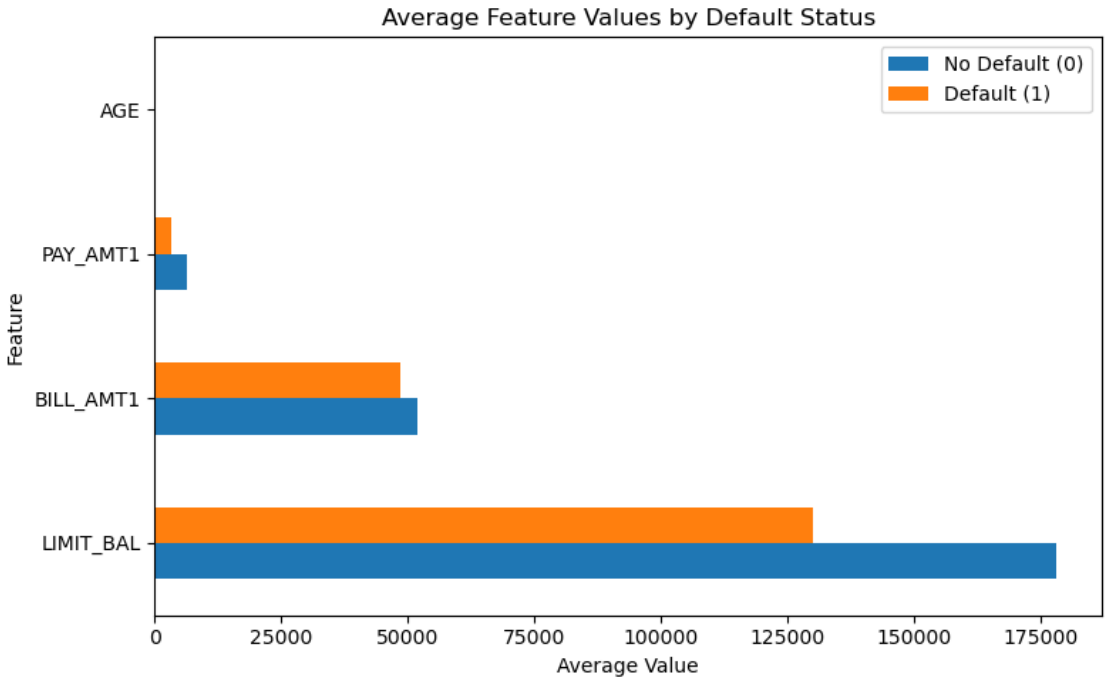

LIMIT_BAL Distribution (Non-default)
Count
2500
2000
1500
1000
500
0
0.0
0.2
0.4
0.6
0.8
1.0
LIMIT_BAL
1e6



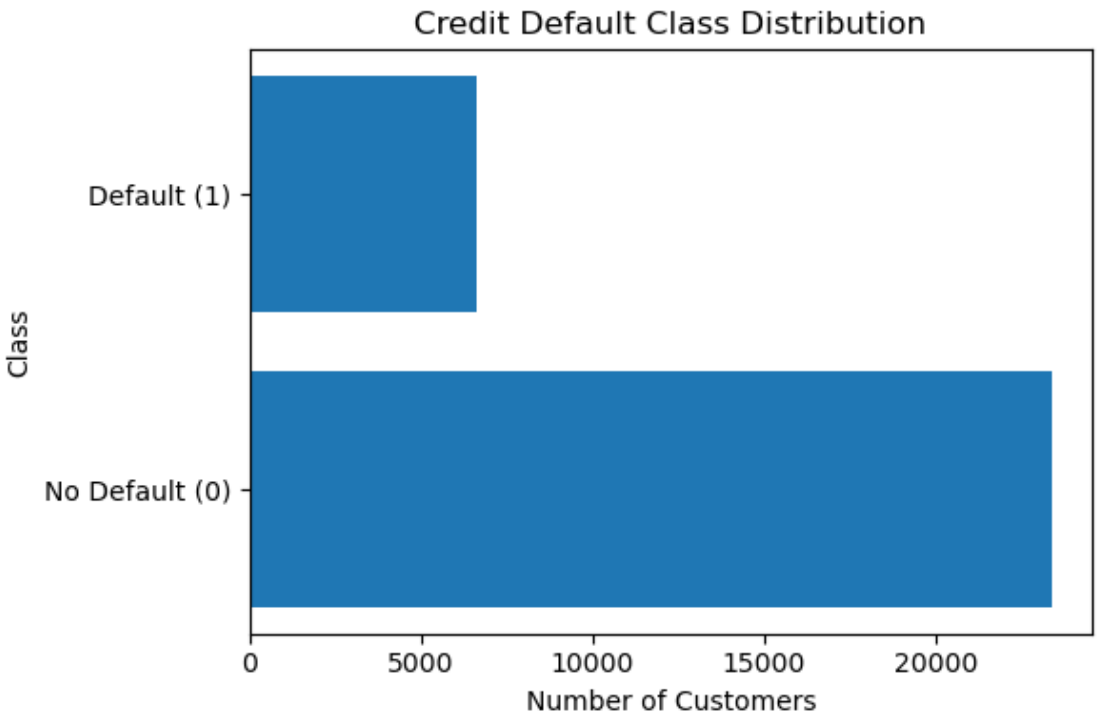

[26]:
```
# 6c) Correlation heatmap for numeric columns
num_df = df.select_dtypes(include=[np.number])
corr = num_df.corr(numeric_only=True)

plt.figure(figsize=(10, 8))
plt.imshow(corr, aspect="auto")
plt.title("Correlation Heatmap (Numeric Features)")
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90, fontsize=7)
plt.yticks(range(len(corr.columns)), corr.columns, fontsize=7)
plt.tight_layout()
plt.show()
```




[27]:
```
#6d Horizontal bar chart: average feature values by default status

features_to_compare = ["LIMIT_BAL", "BILL_AMT1", "PAY_AMT1", "AGE"]
features_to_compare = [f for f in features_to_compare if f in df.columns]

group_means = df.groupby(target_col)[features_to_compare].mean()

group_means.T.plot(kind="barh", figsize=(8, 5))
plt.title("Average Feature Values by Default Status")
plt.xlabel("Average Value")
plt.ylabel("Feature")
plt.legend(["No Default (0)", "Default (1)"])
plt.tight_layout()
plt.show()
```




[28]:
```
class_counts = df[target_col].value_counts().sort_index()

plt.figure(figsize=(6, 4))
plt.barh(["No Default (0)", "Default (1)"], class_counts.values)
plt.title("Credit Default Class Distribution")
plt.xlabel("Number of Customers")
plt.ylabel("Class")
plt.tight_layout()
plt.show()
```




This is just my experimentation with horizontal bar charts.

### 7. Train/Test Split

[30]:
```
# Stratification preserves the imbalance ratio.

from sklearn.model_selection import train_test_split

X = df.drop(columns=[target_col])
y = df[target_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training shape:", X_train.shape)
print("Testing shape:", X_test.shape)
print("Training class distribution:")
print(y_train.value_counts(normalize=True))
```

```
Training shape: (22500, 23)
Testing shape: (7500, 23)
```

```
Training class distribution:
default.payment.next.month
0    0.7788
1    0.2212
Name: proportion, dtype: float64
```

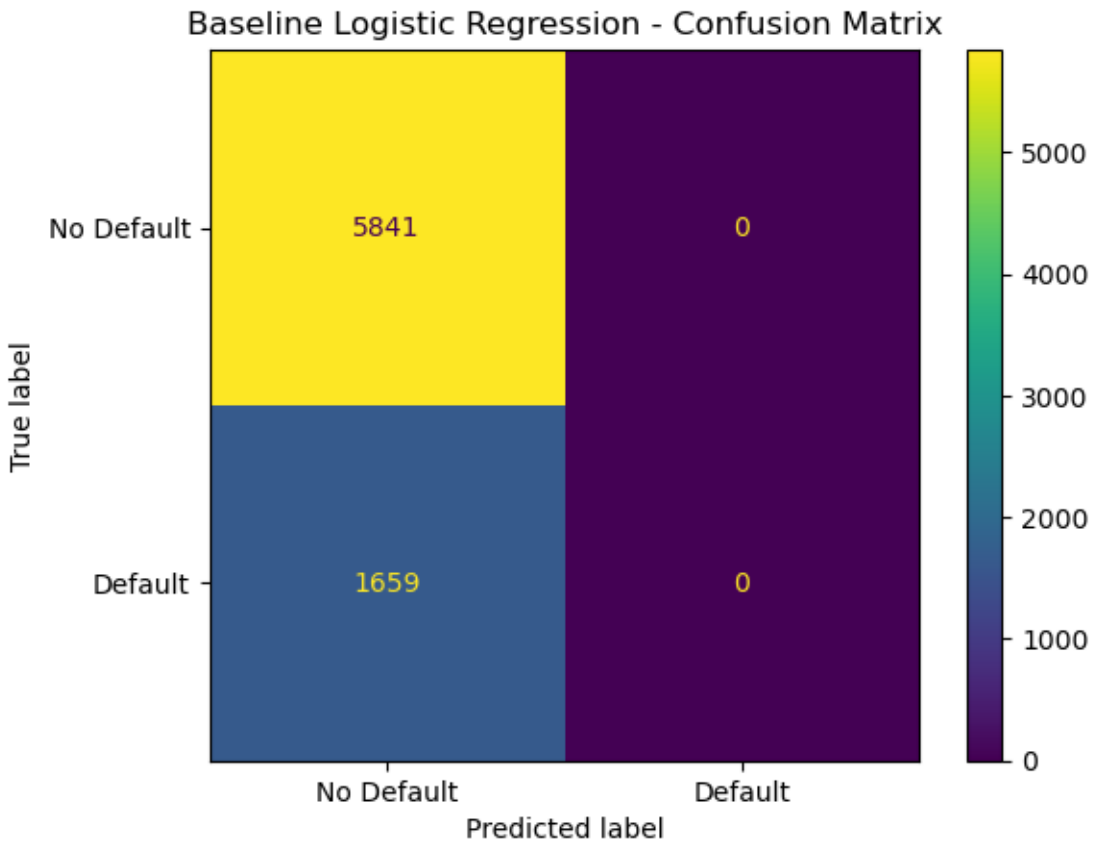
I split the dataset so I can evaluate performance on unseen data.

### 8. Build Baseline Logistic Regression Model

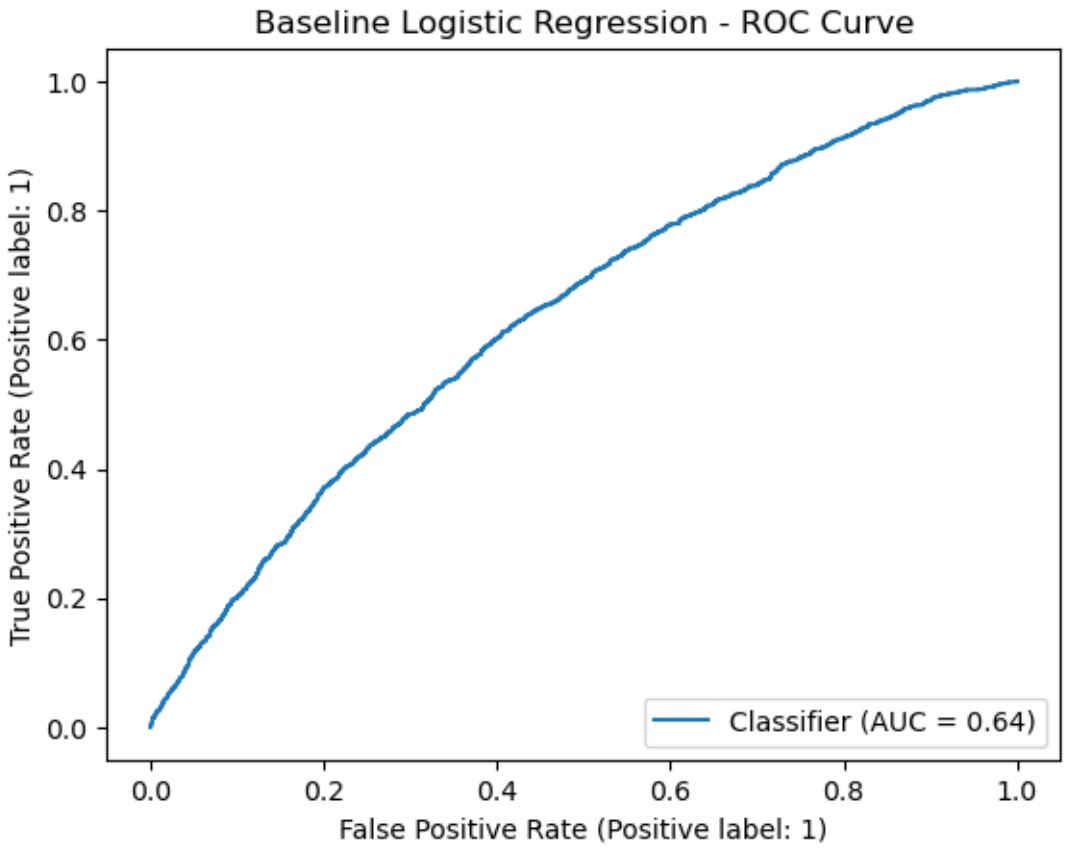
[31]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix,␣
 ↪ConfusionMatrixDisplay, roc_auc_score, RocCurveDisplay

log_reg = LogisticRegression(max_iter=2000)

log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)
y_proba = log_reg.predict_proba(X_test)[:, 1]

print("Baseline Logistic Regression")
print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=["No Default", "Default"]).
 ↪plot(values_format="d")
plt.title("Baseline Logistic Regression - Confusion Matrix")
plt.show()

auc = roc_auc_score(y_test, y_proba)
print("ROC-AUC:", round(auc, 4))

RocCurveDisplay.from_predictions(y_test, y_proba)
plt.title("Baseline Logistic Regression - ROC Curve")
plt.show()
```

```
Baseline Logistic Regression
              precision    recall  f1-score   support

           0       0.78      1.00      0.88      5841
           1       0.00      0.00      0.00      1659

    accuracy                           0.78      7500
   macro avg       0.39      0.50      0.44      7500
weighted avg       0.61      0.78      0.68      7500


C:\Users\ayden\miniconda3\envs\book_env\lib\site-
packages\sklearn\metrics\_classification.py:1469: UndefinedMetricWarning:
```

```
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
C:\Users\ayden\miniconda3\envs\book_env\lib\site-
packages\sklearn\metrics\_classification.py:1469: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
C:\Users\ayden\miniconda3\envs\book_env\lib\site-
packages\sklearn\metrics\_classification.py:1469: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
```




```
ROC-AUC: 0.6365
```



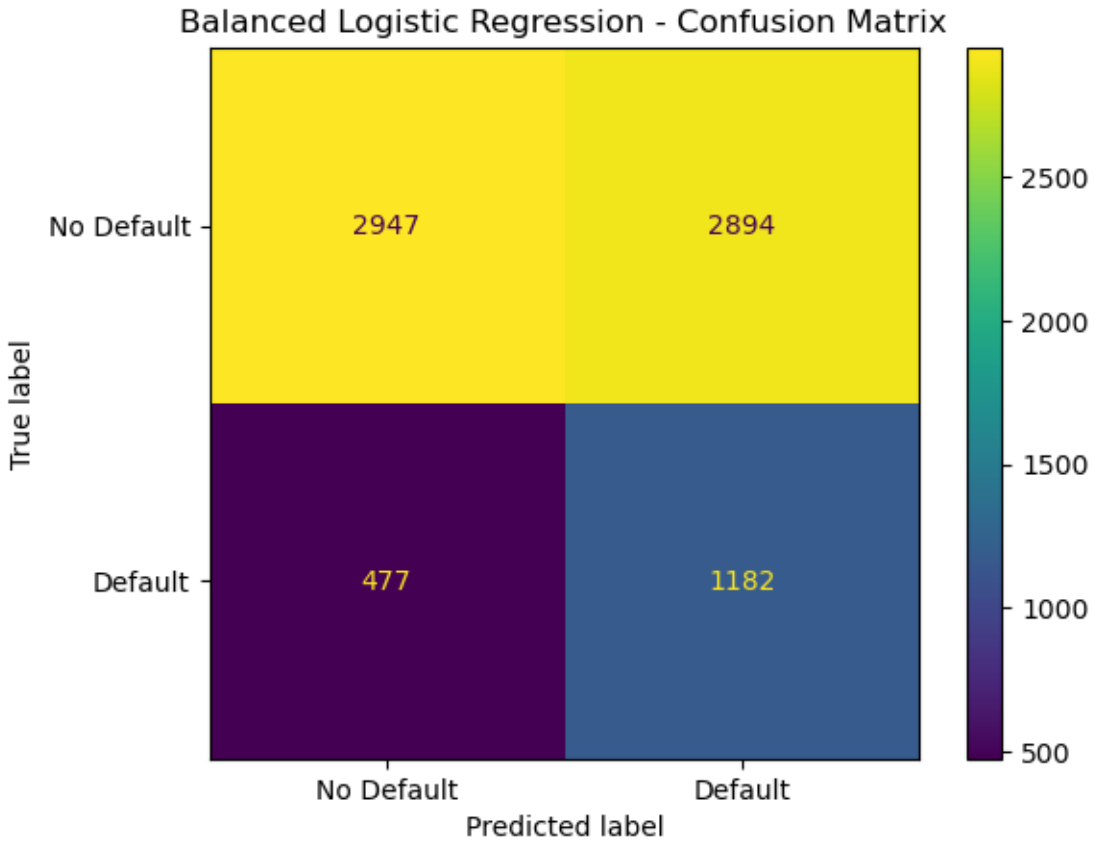

I first build a baseline model without handling imbalance to understand default performance.

### 9. Balanced Logistic Regression

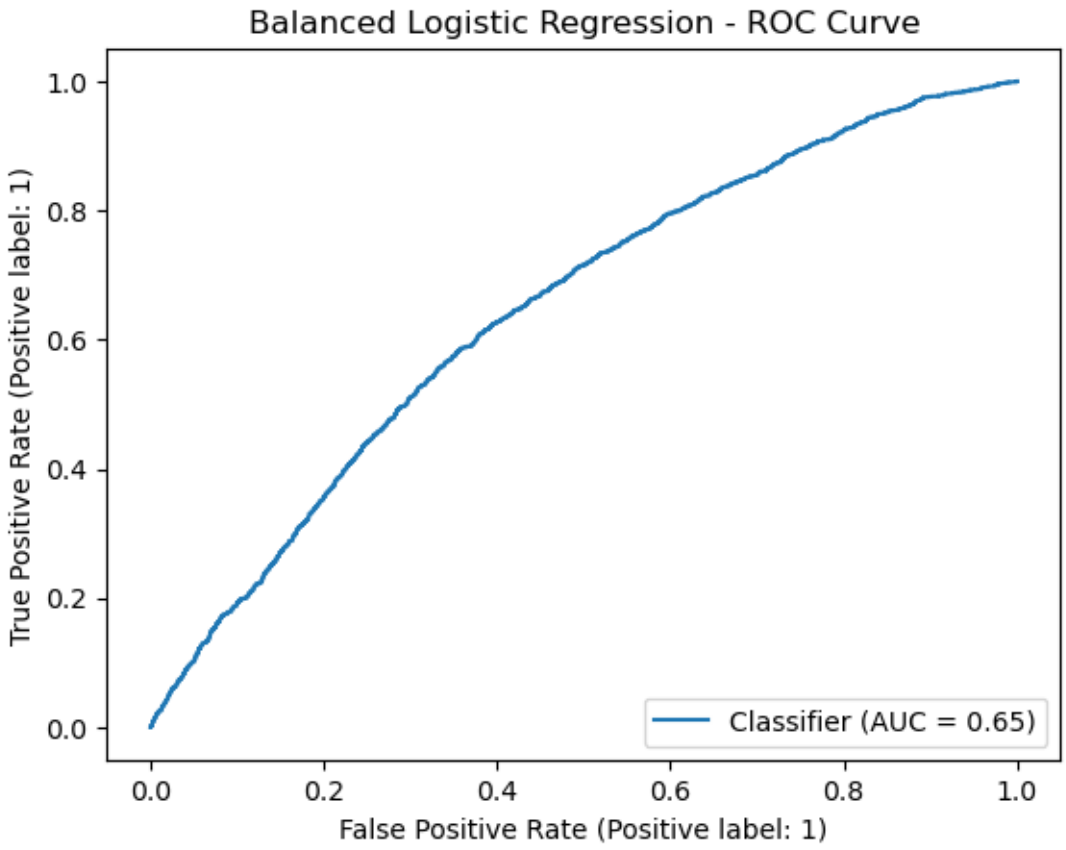
[32]:
```
log_reg_bal = LogisticRegression(max_iter=2000, class_weight="balanced")

log_reg_bal.fit(X_train, y_train)

y_pred_bal = log_reg_bal.predict(X_test)
y_proba_bal = log_reg_bal.predict_proba(X_test)[:, 1]

print("Balanced Logistic Regression")
print(classification_report(y_test, y_pred_bal))

cm_bal = confusion_matrix(y_test, y_pred_bal)
ConfusionMatrixDisplay(cm_bal, display_labels=["No Default", "Default"]).
 ↪plot(values_format="d")
plt.title("Balanced Logistic Regression - Confusion Matrix")
plt.show()

auc_bal = roc_auc_score(y_test, y_proba_bal)
```

```
print("ROC-AUC:", round(auc_bal, 4))

RocCurveDisplay.from_predictions(y_test, y_proba_bal)
plt.title("Balanced Logistic Regression - ROC Curve")
plt.show()
```

```
Balanced Logistic Regression
              precision    recall  f1-score   support

           0       0.86      0.50      0.64      5841
           1       0.29      0.71      0.41      1659

    accuracy                           0.55      7500
   macro avg       0.58      0.61      0.52      7500
weighted avg       0.73      0.55      0.59      7500
```




```
ROC-AUC: 0.6463
```




Because the dataset is imbalanced, I used class_weight='balanced' so that the model gives more importance to default cases.

[ ]: